GUION DE DEFENSA ORAL (15 MINUTOS) — PARTE 1 (Minutos 00:00 a 05:00)

[00:00 - 02:30] BLOQUE 1 (2 min 30 s) — ORADOR: TOBÍAS CARBALLO

Tema: Presentación del Problema, Representación RGB y Algoritmo Cartoon (Rama 1 y Rama 2)

Mostrar en pantalla: Informe Word: Figuras 1, 2 y 2b (foto1.jpg, foto2.jpg, foto3.jpg)

Buenas tardes profesora. Con mi compañero Santino Dacuy vamos a presentar nuestro Trabajo Práctico Final Integrador de Computación Avanzada, que consiste en el diseño, paralelización y análisis de desempeño de un generador de imágenes estilo Cartoon desarrollado íntegramente en lenguaje C (estándar C11) bajo cuatro paradigmas: Secuencial, Memoria Compartida con OpenMP, Memoria Distribuida con MPI y Computación Paralela Híbrida (MPI + OpenMP).

Respecto a la representación de datos: toda imagen digital RGB de ancho W por alto H se almacena en memoria como un arreglo unidimensional contiguo ordenado por filas (row-major order) de W * H * 3 bytes. Esto maximiza la localidad espacial en la caché L1/L2 y permite transferir bloques contiguos de filas directamente por MPI. Además, cumpliendo la consigna, cuando una máscara de convolución accede fuera de los bordes de la imagen, adoptamos el color neutro negro (0, 0, 0).

¿Cómo transforma el algoritmo una imagen real en un Cartoon? Como vemos en la Figura 1 y en las pruebas con nuestras fotografías (foto1.jpg, foto2.jpg y foto3.jpg), el pipeline se divide en dos ramas independientes que luego se suman:

• RAMA 1 — Detección de Bordes (3 pasos obligatorios del apunte): (1) Filtrado o Borroneo mediante convolución con máscara pasa-bajo de media aritmética de 3x3 (peso 1/9) o 5x5 (peso 1/25) para eliminar el ruido; (2) Resaltado de bordes aproximando el gradiente 2D con los operadores direccionales de Sobel (Gx y Gy en 3x3 o 5x5), calculando la magnitud |G| = sqrt(Gx^2 + Gy^2) ponderada por luminancia (0.30 R + 0.59 G + 0.11 B); y (3) Detección mediante Umbralado (Umbralize), que asigna 255 (blanco) si el gradiente supera el umbral o 0 (negro) en caso contrario.

• RAMA 2 — Posterizado por Canal de Color: Cuantiza cada canal (R, G y B) reduciendo los 256 tonos continuos a 3 o 9 rangos discretos mediante una tabla LUT precalculada en O(1), y suma los tres canales en una sola imagen.

• COMBINACIÓN FINAL: Suma píxel a píxel con saturación en 255 la imagen de bordes y la imagen posterizada.

[02:30 - 05:00] BLOQUE 2 (2 min 30 s) — ORADOR: SANTINO DACUY

Tema: Marco Teórico de Computación Paralela Híbrida y Núcleo Secuencial (cartoon_core.h y cartoon_seq.c)

Mostrar en pantalla: Editor de código mostrando include/cartoon_core.h y src/cartoon_seq.c

Gracias Tobías. Antes de ver cada versión paralela, introduzco el marco teórico de la Computación Paralela Híbrida. Las arquitecturas actuales de alto rendimiento son jerárquicas: combinan múltiples nodos interconectados por red (memoria distribuida) donde cada nodo posee internamente procesadores multi-core que comparten memoria RAM y caché (memoria compartida).

El modelo híbrido utiliza MPI para el paralelismo de grano grueso entre procesos independientes y OpenMP para el paralelismo de grano fino entre los hilos internos de cada proceso. ¿Cuáles son sus ventajas? Reduce drásticamente el overhead de comunicación y la duplicación de buffers fantasma: en vez de lanzar 16 procesos MPI aislados, podemos lanzar 2 o 4 procesos MPI con 4 hilos OpenMP cada uno, compartiendo las cachés L2/L3. Como desventaja o desafío, exige coordinar dos niveles de paralelismo y configurar el nivel de soporte de hilos en MPI.

Para garantizar que la comparativa de Speedup y Eficiencia entre las 4 versiones sea 100% justa, programamos el módulo include/cartoon_core.h, donde están las funciones matemáticas que comparten los cuatro ejecutables operando por rango de filas [fila_ini, fila_fin): paso1_filtrado_borroneo_rango(), paso2y3_resaltado_y_deteccion_rango(), rama2_posterizado_rango() y combinar_bordes_y_posterizado_rango().

En el Inciso 1 (src/cartoon_seq.c), ejecutamos secuencialmente estas cuatro funciones sobre las H filas completas y medimos el tiempo de cómputo puro Ts (sin contar lectura/escritura de disco) con temporizadores de alta precisión.

GUION DE DEFENSA ORAL (15 MINUTOS) — PARTE 2 (Minutos 05:00 a 10:30)

[05:00 - 07:30] BLOQUE 3 (2 min 30 s) — ORADOR: TOBÍAS CARBALLO

Tema: Memoria Compartida con OpenMP (src/cartoon_omp.c — Inciso 3)

Mostrar en pantalla: Código src/cartoon_omp.c (líneas 75-135) + Gráfico openmp_speedup_eficiencia.png

Pasando al Inciso 3 (src/cartoon_omp.c), implementamos la versión en Memoria Compartida con OpenMP en C. Aquí toda la imagen reside en el espacio de direcciones único del proceso.

Aplicamos descomposición de datos por filas sobre el bucle externo (y = 0 hasta H-1) mediante la directiva '#pragma omp parallel for default(none) schedule(static)'. Elegimos planificación estática (schedule(static)) porque el costo aritmético de aplicar la máscara de convolución y la tabla de posterizado es exactamente idéntico para todas las filas de la imagen. Así dividimos el trabajo en bloques contiguos de H/T filas con cero overhead de planificación en tiempo de ejecución y máxima localidad espacial en caché.

¿Cómo garantizamos ausencia de condiciones de carrera (data races)? En cada fase, el buffer de entrada es estrictamente de solo lectura (const uint8_t *) y cada hilo escribe únicamente en sus propias filas asignadas del buffer de salida, sin solapamiento.

Respecto a las dependencias entre etapas: para que el Paso 2 (Sobel) pueda leer las filas vecinas (y-R .. y+R) de la imagen borroneada (img_blur), es obligatorio que el Paso 1 haya finalizado en toda la imagen. Esto lo garantiza la barrera implícita al cierre de la primera región '#pragma omp parallel for'. Luego, Sobel, Posterizado y Suma se ejecutan sin conflictos.

[07:30 - 10:30] BLOQUE 4 (3 min 00 s) — ORADOR: SANTINO DACUY

Tema: Memoria Distribuida con MPI y Filas Fantasma / Halo Rows (src/cartoon_mpi.c — Inciso 2)

Mostrar en pantalla: Código src/cartoon_mpi.c (función intercambiar_halos) + Gráfico mpi_speedup_eficiencia.png

En el Inciso 2 (src/cartoon_mpi.c), desarrollamos la versión de Memoria Distribuida con MPI. Aplicamos descomposición de dominio 1D por bloques de filas equilibrados. Como la altura H de una imagen no siempre es divisible por P procesos (por ejemplo, foto1.jpg tiene 906 filas y foto2.jpg tiene 573 filas), calculamos los arreglos sendcounts y displs repartiendo el resto H % P entre los primeros procesos y distribuimos la imagen desde el rank 0 con MPI_Scatterv.

Aquí surge el problema clave de memoria distribuida: cuando un proceso aplica la máscara de 3x3 (radio R=1) o 5x5 (radio R=2) sobre su primera fila local o su última fila local, necesita leer R filas que están en la memoria del proceso vecino superior (rank - 1) y R filas del proceso vecino inferior (rank + 1).

Lo resolvimos implementando la técnica de Filas Fantasma (Halo / Ghost Rows): cada proceso reserva memoria para (m_k + 2R) filas. Las primeras R filas alojan el halo superior, las m_k centrales contienen las filas propias, y las últimas R alojan el halo inferior.

Programamos la función intercambiar_halos() usando MPI_Sendrecv para enviar nuestras filas de frontera y recibir las filas fantasma de rank - 1 y rank + 1 en una sola operación acoplada libre de deadlocks. En los bordes globales de la imagen (rank 0 hacia arriba y rank P-1 hacia abajo), usamos la constante MPI_PROC_NULL: así MPI convierte esa comunicación en un no-op y las filas fantasma externas conservan los ceros de calloc, aplicando automáticamente el borde negro (0, 0, 0).

Realizamos dos intercambios de halos: el primero sobre local_orig antes del Borroneo, y el segundo sobre local_blur antes de Sobel. El Posterizado y la Suma son locales y finalmente recolectamos la imagen con MPI_Gatherv.

GUION DE DEFENSA ORAL (15 MINUTOS) — PARTE 3 (Minutos 10:30 a 15:00)

[10:30 - 13:00] BLOQUE 5 (2 min 30 s) — ORADOR: TOBÍAS CARBALLO

Tema: Versión Híbrida MPI + OpenMP (src/cartoon_hybrid.c — Inciso 4) y Demo en Vivo

Mostrar en pantalla: Código src/cartoon_hybrid.c + Terminal PowerShell ejecutando en vivo

En el Inciso 4 (src/cartoon_hybrid.c), unificamos ambos modelos en la versión Híbrida MPI + OpenMP. Inicializamos el entorno con MPI_Init_thread(..., MPI_THREAD_FUNNELED, ...): el hilo maestro de cada proceso MPI realiza las comunicaciones colectivas y el intercambio de filas fantasma fuera de las regiones paralelas, mientras que dentro de cada proceso se despliegan T hilos OpenMP para procesar las m_k filas locales.

Además, en la versión híbrida aplicamos Fusión de Bucles (Loop Fusion): luego del segundo intercambio de halos, un único bloque '#pragma omp parallel for schedule(static)' ejecuta en una sola pasada por fila el cálculo de Sobel+Umbral, el Posterizado RGB y la Suma final, reutilizando los datos calientes en la caché L1/L2.

[DEMO EN VIVO — 11:45 a 13:00]: Ahora ejecutamos en la terminal sobre nuestras fotos reales (foto1.jpg, foto2.jpg y foto3.jpg) las versiones Secuencial (con --etapas), OpenMP e Híbrida, comprobando que tardan milisegundos y generan exactamente el mismo resultado visual (MaxDiff = 0).

[13:00 - 15:00] BLOQUE 6 (2 min 00 s) — ORADOR: SANTINO DACUY

Tema: Análisis de Desempeño (Inciso 5): Speedup, Eficiencia, Filtros y Conclusiones

Mostrar en pantalla: Gráficos comparativa_paradigmas_5000.png e impacto_filtros_y_comunicacion.png

Finalmente, en el Inciso 5 evaluamos las tres resoluciones exigidas (800x800, 2000x2000 y 5000x5000) variando filtros (3x3 y 5x5) y posterizados (3 y 9 rangos) en un Ryzen 7 5700U (8 núcleos físicos / 16 hilos lógicos SMT), habiendo verificado antes que las 4 versiones dan Diferencia Máxima = 0 píxel a píxel.

1. Escalamiento con la resolución: En 800x800 el tiempo secuencial es tan bajo (28 ms) que el overhead limita el Speedup a 5.25x. Pero en 5000x5000 (25 millones de píxeles), el cómputo domina y alcanzamos un Speedup de 8.07x en OpenMP (eficiencia de hasta 94% en 4 hilos) y 7.71x en la versión Híbrida 4Px4T con filtro 5x5 (bajando el tiempo de 7.20 s a 0.93 s).

2. Impacto de filtro (3x3 vs 5x5) y posterizado (3 vs 9): El filtro 5x5 realiza 25 lecturas por píxel frente a 9 del 3x3 (2.78x más trabajo teórico), elevando el tiempo medido en ~2.57x y mejorando la eficiencia paralela. En cambio, pasar de 3 a 9 rangos de posterizado no cambia el tiempo porque se resuelve en O(1) mediante la tabla LUT.

3. Núcleos físicos vs SMT y ventaja Híbrida: Hasta 8 workers (1 por núcleo físico) la eficiencia se sostiene entre 73% y 94%; al pasar a 16 workers (SMT) el Speedup sigue creciendo pero la eficiencia baja al ~48% por compartición de unidades físicas. En el caso más exigente (5000x5000, filtro 5x5), la versión Híbrida supera tanto a MPI puro como a OpenMP puro, con menos del 7% de tiempo de comunicación. ¡Muchas gracias!

GUION DE DEFENSA ORAL — PARTE 4 (Comandos Demo en Vivo y Preguntas de Examen)

1. COMANDOS LISTOS PARA COPIAR Y PEGAR EN LA DEMO EN VIVO (BLOQUE 5 — Minuto 11:45):

```
# Configurar PATH de compilador GCC y Microsoft MPI en la consola PowerShell:
$env:PATH = "C:\tmp\toolchain\mingw64\bin;C:\tmp\toolchain\msmpi\runtime_1;" + $env:PATH
cd "c:\Users\tobic\OneDrive\Documentos\Facu\4to Año Sistemas\Computación Avanzada\TPs\TP-Final"

# A) Ejecutar versión Secuencial sobre foto2.jpg exportando las etapas (--etapas):
.\bin\cartoon_seq.exe foto2.jpg img\output\demo_seq.png 5 9 75 --etapas

# B) Ejecutar versión Memoria Compartida (OpenMP) con 8 hilos sobre foto1.jpg o foto2.jpg:
.\bin\cartoon_omp.exe foto2.jpg img\output\demo_omp.png 5 9 8 75

# C) Ejecutar versión Memoria Distribuida (MPI) con 4 procesos:
mpiexec -n 4 .\bin\cartoon_mpi.exe foto2.jpg img\output\demo_mpi.png 5 9 75

# D) Ejecutar versión Paralela Híbrida (2 procesos MPI x 4 hilos OpenMP = 8 workers):
mpiexec -n 2 .\bin\cartoon_hybrid.exe foto2.jpg img\output\demo_hyb.png 5 9 4 75
```

2. BANCO DE PREGUNTAS Y RESPUESTAS CLAVE PARA LA DEFENSA ORAL (Q&A):

1. ¿Por qué hacen DOS intercambios de filas fantasma (halos) en MPI e Híbrido?

Respuesta: Porque hay dos convoluciones sucesivas: el Paso 1 (Borroneo) necesita las R filas vecinas de la imagen original (local_orig), pero el Paso 2 (Sobel) opera sobre la imagen YA borroneada (local_blur). Como cada proceso solo borronea sus m_k filas propias, debe intercambiar las fronteras de local_blur antes de ejecutar Sobel.

2. ¿Cómo trataron los bordes exteriores de la imagen cuando la máscara 3x3 o 5x5 sobresale?

Respuesta: Con el valor neutro negro (0,0,0) pedido en el enunciado. En Secuencial/OpenMP la función obtener_pixel_con_borde_negro() retorna 0 fuera de [0,H)x[0,W). En MPI/Híbrido usamos MPI_PROC_NULL en el borde superior del rank 0 e inferior del rank P-1 sobre memoria inicializada con calloc (ceros).

3. ¿Por qué eligieron MPI_Scatterv / MPI_Gatherv en vez de MPI_Scatter / MPI_Gather?

Respuesta: Porque MPI_Scatter exige que la altura H sea múltiplo exacto de P. Con MPI_Scatterv/Gatherv soportamos cualquier imagen (como foto1.jpg de 906 filas o foto2.jpg de 573 filas con 4 u 8 procesos) repartiendo el resto H % P equitativamente.

4. ¿Qué significa MPI_THREAD_FUNNELED en la versión Híbrida?

Respuesta: Indica al entorno MPI que el proceso es multihilo (OpenMP), pero que únicamente el hilo maestro realizará llamadas a funciones MPI, siempre fuera de las regiones paralelas de OpenMP.